Caçadores de Crateras: detecção e caracterização de crateras lunares em imagens de satélite, com interface gráfica

Disciplina: Processamento de Imagens (UFRPE) - Lucas Fernando da Silva Cambuim

Integrantes: Gabriel Barros de Morais, Elton da Costa Oliveira e Júlia Karolyne Falcão Diniz Oliveira.

1. Descrição resumida do problema

Satélites da NASA fotografam continuamente a superfície da Lua. Nessas imagens, as crateras de impacto são uma das feições mais importantes: a contagem e o tamanho das crateras permitem estimar a idade relativa de uma superfície, e as crateras também servem como pontos de referência para a navegação de sondas e módulos de pouso sem GPS. Marcar crateras manualmente, porém, é lento e sujeito a erros.

O objetivo deste projeto é desenvolver um programa em Python, com interface gráfica, em que o usuário anexa um recorte de imagem da Lua em escala de cinza e vê as crateras destacadas na própria imagem. Para cada cratera encontrada, o programa calcula a **posição do centro, o raio, a área, o perímetro, a circularidade e a máscara binária do contorno**. O resultado será comparado com marcações manuais (ground truth) para avaliar a qualidade da detecção.

O problema é desafiador porque as imagens têm sombras muito escuras dentro das crateras, bordas voltadas ao sol com **brilho excessivo**, solo com muita textura e ruído, e crateras de tamanhos muito diferentes, algumas sobrepostas ou parcialmente erodidas.

2. Base de dados

- **Origem:** imagens públicas da NASA disponíveis no Planetary Data System (PDS), feitas pela câmera LROC da sonda Lunar Reconnaissance Orbiter, que fotografa a superfície da Lua.
- **Formato e resolução:** imagens em escala de cinza de 8 bits, divididas em recortes de 256 a 512 pixels de lado.
- **Cenários de teste:** recortes com crateras isoladas, recortes com muitas crateras e recortes com terreno rugoso ou iluminação rasante, todos da Lua. A quantidade exata de imagens e a marcação manual de referência serão definidas na fase inicial do desenvolvimento.

3. Principais técnicas e justificativas

O projeto será um pipeline clássico de processamento de imagens, com cada etapa justificada pelo problema:

Etapa	Técnica	Justificativa
1. Aquisição	Leitura do recorte em escala de cinza de 8 bits	As imagens planetárias são monocromáticas; os recortes reduzem o custo computacional.

Etapa	Técnica	Justificativa
2. Melhoria de contraste	CLAHE (equalização adaptativa de histograma com limite de contraste)	Sombras escuras dentro das crateras e bordas estouradas ao sol exigem contraste ajustado por região, sem amplificar o ruído como a equalização global faria.
3. Suavização	Filtro Gaussiano	A textura do solo gera falsos alarmes na detecção de bordas; um leve desfoque reduz esse ruído antes da derivada.
4. Detecção de bordas	Sobel e Laplaciano	Realçam as mudanças bruscas de intensidade nas paredes e sombras das crateras.
5. Morfologia matemática	Abertura e fechamento com elemento estruturante em forma de disco	A abertura remove pontos isolados e o fechamento une bordas interrompidas; o disco é adequado porque as crateras são aproximadamente circulares.
6. Segmentação	Limiarização de Otsu e Transformada de Hough para círculos	Otsu escolhe o limiar automaticamente, sem ajuste manual. A Hough vota nos centros e raios mais prováveis e lida bem com bordas incompletas.
7. Descrição e representação	Centro (a, b), raio r, área, perímetro e circularidade	Quantificam cada cratera detectada e permitem descartar regiões pouco circulares.
8. Avaliação	IoU, precisão, recall e tempo de execução	Medem a sobreposição com a marcação manual, os falsos alarmes, as crateras esquecidas e se o processamento é rápido o bastante para a interface responder sem travar.
9. Interface gráfica	Tkinter (Python)	O usuário anexa a foto, o pipeline roda por trás e as crateras são desenhadas sobre a imagem original, com as medidas de cada uma ao lado.

Relação com os conteúdos da disciplina: o projeto contempla Fundamentos do Processamento de Imagens, Melhorias em Imagens e Filtragem (CLAHE e Gaussiano), Descrição e Representação de Imagens (características geométricas), Morfologia Matemática e Segmentação de Imagens (bordas, Otsu e Hough).

Implementação: Python, com as bibliotecas obrigatórias OpenCV e NumPy e as opcionais SciPy (`scipy.ndimage`) e Matplotlib, organizada em módulos (`preprocessing.py`, `segmentation.py`, `metrics.py` e `gui.py`) com comentários e documentação básica. A interface será feita só em Python, com Tkinter (já vem com o Python) e Pillow para exibir as imagens. O pipeline fica separado da interface, então também pode ser rodado sozinho em um notebook para testes. Se o prazo permitir, o visual pode ser melhorado com CustomTkinter. Sempre que possível, implementaremos algumas etapas do zero (por exemplo, a convolução e os filtros).

Escopo: o projeto usa apenas técnicas clássicas vistas na disciplina, com um pouco de visão computacional (Transformada de Hough para círculos). Conforme o andamento, avaliaremos extras simples, como descartar candidatos pouco circulares, comparar a suavização Gaussiana com um filtro passa-baixa no domínio da frequência (Transformada de Fourier) ou até implementar algo relacionado ao aprendizado de máquina.

4. Entregáveis previstos

- Código-fonte completo em Python, incluindo a interface gráfica, com tutorial de execução dos experimentos (repositório no GitHub).
- Relatório ou slides com introdução, fundamentação teórica, metodologia, experimentos, resultados e conclusão, incluindo imagens intermediárias, comparações antes/depois e análise crítica.
- Vídeo de apresentação de até 10 minutos com a motivação, o funcionamento da solução (incluindo a interface), as principais técnicas e a demonstração dos resultados.

5. Referências iniciais

1. GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing*. Pearson.
2. DUDA, R. O.; HART, P. E. Use of the Hough transformation to detect lines and curves in pictures. *Communications of the ACM*, 1972.
3. ROBBINS, S. J. A new global database of lunar impact craters >1–2 km. *Journal of Geophysical Research: Planets*, 2019.
4. ROBINSON, M. S. et al. Lunar Reconnaissance Orbiter Camera (LROC) instrument overview. *Space Science Reviews*, 2010.
5. NASA Planetary Data System, Imaging Node: https://pds-imaging.jpl.nasa.gov/ (imagens LROC).
6. Documentação do OpenCV: https://docs.opencv.org/ (CLAHE, filtros, morfologia, Otsu e `HoughCircles`).
7. Documentação do Python sobre Tkinter: https://docs.python.org/3/library/tkinter.html (interface gráfica).